SafeNav Phase 1: Comprehensive Static Analysis Architecture and Implementation Report

1. Executive Overview and Architectural Philosophy

1.1 The Imperative for Advanced Static Analysis

In the contemporary cybersecurity landscape, the proliferation of malicious Uniform Resource Locators (URLs) constitutes one of the most pervasive vectors for compromise. Phishing campaigns, drive-by downloads, and social engineering attacks increasingly rely on ephemeral, sophisticated link structures to evade detection. The SafeNav project addresses this challenge through a bifurcated architecture, separating high-speed static analysis from resource-intensive dynamic sandboxing. This report provides an exhaustive technical specification for **Phase 1: The Static Analysis Engine**, which serves as the system's primary filtration layer.

The operational objective of Phase 1 is to maximize threat detection throughput while minimizing latency. Industry data suggests that a significant majority—approximately 90%—of malicious links can be identified through rigorous inspection of their syntactic, registration, and cryptographic metadata without the need for full browser execution.[1] By filtering these threats at the static layer, the system preserves expensive computational resources for the "Tier 2" deep scan, which is reserved for ambiguous or highly obfuscated targets.

1.2 Architectural Principles: The "Fail-Fast" Doctrine

The design of the Static Analysis Engine is governed by the "Fail-Fast" doctrine. In high-volume security scanning, latency is a critical performance metric. Users expect near-instantaneous feedback on link safety. Consequently, Phase 1 is engineered to execute within a time window of milliseconds to a few seconds.[1] This constraint mandates the exclusion of any processes that require rendering a Document Object Model (DOM), executing JavaScript, or initiating heavy virtual machine processes.

Instead, the architecture relies on a pipeline of lightweight, deterministic, and probabilistic checks. These checks operate on the "surface" of the URL—analyzing the string itself, the DNS records associated with it, the SSL/TLS certificate chain presented by the server, and the immediate HTTP response headers. This approach ensures scalability, allowing the SafeNav system to handle high concurrency rates typical of enterprise network traffic or public-facing security tools.[1]

1.3 Scope of Analysis

This report details the implementation logic for eight distinct analytical modules that comprise Phase 1. It moves beyond high-level descriptions to provide the algorithmic logic, edge-case handling, and technological specifications required for deployment. Furthermore, it defines a novel, weighted risk scoring algorithm that synthesizes disparate signals—such as domain age, lexical entropy, and certificate validation—into a coherent, explainable safety verdict.

---

2. Technological Stack and Infrastructure Specification

To meet the rigorous performance and accuracy requirements of Phase 1, the technology stack has been selected based on criteria of execution speed, library maturity, and support for asynchronous operations.

2.1 Core Programming Language: Python 3.11+

Python is selected as the primary development language due to its dominance in the cybersecurity and data science domains. While languages like Go or Rust offer superior raw execution speed, Python's ecosystem for string manipulation (urllib, regex), network analysis (dnspython, ssl), and machine learning (scikit-learn) is unparalleled.[2] The potential performance overhead of Python is mitigated by utilizing its asynchronous features and highly optimized C-extensions for critical libraries.

2.2 Web Framework: FastAPI (Asynchronous Architecture)

The backbone of the scanning engine requires a non-blocking I/O model to handle multiple concurrent URL scans. **FastAPI** is the designated framework, running on an ASGI (Asynchronous Server Gateway Interface) server such as Uvicorn. Unlike synchronous frameworks (e.g., Flask), FastAPI allows the system to initiate multiple network requests—such as WHOIS lookups or HTTP header checks—in parallel without blocking the main execution thread.[4] This is crucial for Phase 1, where network latency (waiting for external servers) is the primary bottleneck.

2.3 Machine Learning Infrastructure: Scikit-Learn

For the predictive risk modeling component, **Scikit-learn** is chosen over deep learning frameworks like TensorFlow or PyTorch. The nature of URL analysis involves tabular feature data (e.g., length of URL, count of dots, age of domain) rather than unstructured perceptual data (images/audio). Decision-tree-based ensembles, specifically **Random Forest**, have proven empirically superior for this type of classification in terms of training speed, inference

latency, and crucially, feature interpretability.[3]

2.4 Containerization and Deployment

To ensure consistency across development and production environments, the engine is containerized using **Docker**. This encapsulates the Python runtime, system-level dependencies (such as OpenSSL libraries), and ML models into a portable unit. Orchestration via Docker Compose or Kubernetes allows for horizontal scalability, enabling the system to spin up additional static analysis nodes during peak load.

---

3. Module I: Link Intake, Sanitization, and Normalization Logic

The first stage of the pipeline is data ingestion. Malicious actors frequently employ obfuscation techniques to disguise the true nature of a URL. Therefore, before any security checks can be performed, the input string must be rigorously sanitized and normalized to a canonical form.

3.1 Detailed Implementation Logic

The normalization process must adhere strictly to RFC 3986 standards while accounting for common evasion techniques.

1. Percent-Decoding and Recursive Sanitation:
   Web browsers automatically decode percent-encoded characters (e.g., %20 for space) before making a request. Security scanners must do the same to see the payload. Attackers utilize "double encoding" (e.g., %2520) to bypass simple filters. The implementation logic requires a recursive decoding loop:
   - *Step A:* Apply urllib.parse.unquote to the string.[2]
   - *Step B:* Compare the result to the input.
   - *Step C:* If they differ, repeat Step A. This loop continues until convergence or a maximum depth (e.g., 5 iterations) is reached to prevent infinite loops.
2. Whitespace and Control Character Stripping:
   Input strings often contain accidental whitespace or malicious invisible control characters designed to break parsers. The system must strip all leading/trailing whitespace and remove non-printable characters (ASCII 0-31) from the string.
3. **Scheme and Authority Standardization:**
   - **Scheme Normalization:** Convert the protocol scheme to lowercase (e.g., HTTP:// becomes http://). If no scheme is provided, default to http:// for initial analysis, though https:// is preferred for connection attempts.
   - **Host Parsing:** Utilize urllib.parse.urlsplit to separate the scheme, network location (netloc), path, query, and fragment. The netloc must be converted to lowercase to

ensure case-insensitive domain matching.[2]

4. Internationalized Domain Name (IDN) Conversion (Punycode):
   A critical defense against homograph attacks—where attackers use look-alike characters from different scripts (e.g., Cyrillic 'a' vs. Latin 'a')—is converting all domains to their Punycode (ASCII) representation. The system must process the hostname using Python's idna encoding codec. For example, a domain appearing as apple.com (using Cyrillic characters) would convert to an xn--... string, instantly revealing it as an impostor.1

3.2 Potential Problems and Mitigations

- **Problem:** *Malformed URL Attacks.* Attackers may supply extremely long strings or invalid IPv6 literals to cause denial-of-service (DoS) via regex catastrophic backtracking or memory exhaustion.
  - *Mitigation:* Implement strict length limits (e.g., 2048 characters) and wrap parsing logic in try-except blocks to handle ValueError exceptions gracefully, flagging such inputs immediately as "Invalid/Suspicious".
- **Problem:** *Mixed-Script Homoglyphs.* Simple Punycode conversion handles non-ASCII characters, but mixed-script spoofing can still be visually confusing.
  - *Mitigation:* Integrate a check that flags domains containing characters from multiple distinct unicode scripts (e.g., Latin mixed with Cyrillic), a strong indicator of malicious intent.

---

4. Module II: Heuristic Link Type Identification and Taxonomy

Once normalized, the URL is analyzed to determine its functional intent. This "Link Fingerprinting" helps the user anticipate what will happen if they click the link—whether it opens a webpage, initiates a download, or launches an application.

4.1 Taxonomy and Detection Logic

The system classifies links into non-mutually exclusive categories using high-performance string matching and regular expressions.

1. **Standard Website:** The default classification if the URL utilizes http or https schemes and points to a valid domain name without specific download extensions.
2. IP-Based Links:
   Legitimate public web services rarely use raw IP addresses (e.g., http://192.168.1.1/login) due to SSL certificate management difficulties and lack of memorability. Phishing kits and malware hosts, however, frequently use them to bypass domain blocklists.
   - *Logic:* The extracted netloc is validated using Python's ipaddress library. If ipaddress.ip_address(host) succeeds, the link is flagged as "IP-Based," triggering a

high-risk penalty.[7]

3. Shortened URLs:
   URL shorteners obscure the final destination, making them a favorite tool for attackers.
   - *Logic:* The domain is compared against a curated database or hash set of known shortener domains (e.g., bit.ly, t.co, goo.gl, is.gd). The tldextract library helps isolate the domain name from subdomains to ensure accurate matching.[9]
4. File Download Links:
   URLs ending in executable or archive extensions pose an immediate malware risk.
   - *Logic:* The path component of the URL is parsed to extract the file extension. This is checked against a blacklist: .exe, .apk, .dmg, .zip, .rar, .scr, .bat, .ps1. Detection of these extensions categorizes the link as a "Direct Download".[1]
5. App Deep Links (Mobile Intents):
   Modern mobile operating systems allow URLs to launch native applications. These can be abused to trigger unauthorized actions or install malware.
   - *Logic:* The system inspects the protocol scheme. Schemes other than http/s (e.g., whatsapp://, zoommtg://, tg://) are flagged.
   - *Android Intent Logic:* For intent:// schemes, the system must parse the fragment identifier (everything after #Intent;) to extract the package and scheme parameters. This allows SafeNav to report exactly which app is being targeted (e.g., "Launches com.evil.app").[11]

4.2 Potential Problems and Mitigations

- **Problem:** *False Negatives via Query Parameters.* A URL like example.com/download?file=malware.exe might be missed if the logic only checks the path ending.
  - *Mitigation:* Extend the logic to inspect the values of query parameters for suspicious extensions, not just the path suffix.
- **Problem:** *Obfuscated IP Addresses.* Attackers may use dword, octal, or hex representations of IP addresses (e.g., http://0x7f000001) which standard libraries might not recognize as IPs but browsers will resolve.
  - *Mitigation:* Ensure the IP detection logic accounts for alternative IP formats or relies on the underlying DNS resolver to normalize the host before classification.

---

5. Module III: Lightweight Redirect Tracing and Destination Resolution

A static scan is incomplete if it analyzes only the entry URL. Phishers use "redirect chains" to bounce users through benign sites (like open redirects on Google or Facebook) before landing on the malicious page. This module traces the path to the final destination without

executing client-side scripts.

5.1 Implementation Logic: The "Headless" Trace

This module utilizes the requests or httpx library to simulate a browser's network stack without the overhead of a rendering engine.

1. User-Agent Masquerading:
   Servers often return different content to bots than to real users. To ensure an accurate trace, the HTTP request headers must include a legitimate User-Agent string (e.g., mimicking the latest Chrome on Windows 10). This prevents the scanner from being blocked by basic anti-bot defenses.13
2. Request Execution:
   The system initiates a HTTP GET request with the stream parameter set to True (stream=True). This allows the system to inspect headers and follow redirects without downloading the potentially large response body, preserving bandwidth and processing time.15
3. Chain Analysis Loop:
   The history attribute of the response object contains the list of all intermediate responses (301, 302, 307 status codes). The logic iterates through this list to build a "Redirect Chain" report:
   - **Hop Count:** Count the total number of redirects. A high number (>3) is suspicious.
   - **Cross-Domain check:** For each hop, compare the domain of the current URL with the next. Transitions between entirely different domains (e.g., google.com -> attacker.xyz) are flagged as "Cross-Domain Redirects," which increase the risk score.[1]
4. Loop and Black Hole Detection:
   Redirect loops (A -> B -> A) can hang the scanner. The logic must implement a max_redirects cap (e.g., 10). If the limit is reached, the scan halts, and the final known URL is treated as the destination, with a warning flag raised.17

5.2 Potential Problems and Mitigations

- **Problem:** *JavaScript and Meta-Refresh Redirects*. Static HTTP requests do not execute JavaScript (window.location) or parse HTML meta tags (<meta http-equiv="refresh"...>).
  - *Mitigation:* While Phase 1 cannot execute JS, it *can* perform a lightweight regex search on the response body for patterns like http-equiv="refresh" or window.location. If found, the system flags the result as "Possible Client-Side Redirect" and strongly recommends a Tier 2 Deep Scan.[18]
- **Problem:** *Cloaking*. Sophisticated attacks check for specific browser headers or behaviors (like mouse movement) which static requests lack.
  - *Mitigation:* This is an inherent limitation of static analysis. The mitigation is to rely on IP reputation and other signals if the content is hidden, or to defer to Phase 2 for behavioral analysis.

6. Module IV: Transport Layer Security (SSL/TLS) Inspection

The presence of a padlock icon (HTTPS) is no longer a guarantee of safety; over 80% of phishing sites now use HTTPS. This module analyzes the *quality* and *context* of the SSL certificate to distinguish between legitimate secure sites and encrypted phishing traps.

6.1 Implementation Logic

The analysis uses Python's ssl and socket modules to perform a handshake and retrieve the server's certificate chain.[19]

1. Connection and Retrieval:
   A secure socket connection is established to port 443 of the destination host. The getpeercert() method retrieves the X.509 certificate data in a dictionary format.
2. Certificate Type Classification (DV/OV/EV):
   The logic parses the subject field of the certificate.
   - **Domain Validated (DV):** If the subject contains only the Common Name (CN) (domain name) without Organization (O) details, it is a DV cert. These are automated, free, and require no identity verification, making them the standard for phishing.
   - **Organization Validated (OV) / Extended Validation (EV):** If the subject contains a verified "Organization" field (e.g., "PayPal, Inc."), it indicates a higher level of trust. Phishers rarely obtain these due to the paper trail required.[20]
3. Issuer Analysis:
   The issuer field is checked against a list of issuers known for providing free, automated certificates (e.g., "R3" for Let's Encrypt, "cPanel"). While legitimate, a DV certificate from a free provider on a login page is a high-risk combination.1
4. Certificate Age Calculation:
   The notBefore field indicates when the certificate was issued. The system calculates the certificate's age: Current Time - notBefore.
   - *Heuristic:* A certificate less than 24-48 hours old is a critical risk indicator for "burn" domains set up for immediate, short-lived attacks.[22]
5. Cipher Suite Analysis:
   Using the sslyze or sslscan libraries (or low-level ssl context queries), the system checks the negotiated cipher suite. The presence of deprecated protocols (SSLv3, TLS 1.0) or weak ciphers (RC4, NULL) indicates a neglected or compromised server infrastructure.23

6.2 Potential Problems and Mitigations

- **Problem:** *SNI (Server Name Indication) Requirements.* Many modern hosts host multiple domains on one IP and require SNI to serve the correct certificate.
  - *Mitigation:* The SSL handshake logic must explicitly include the server_hostname

parameter in the wrap_socket call to ensure SNI compatibility.[19]

- **Problem:** *Self-Signed Certificates.*
  - *Mitigation:* If the handshake fails due to a verification error (self-signed or untrusted root), the system captures the error and flags the SSL status as "Invalid/Untrusted" rather than crashing.

---

7. Module V: Domain Reputation, History, and Trust Signals

A domain's history is one of the most reliable predictors of its future behavior. Legitimate businesses rarely switch domains frequently, while attackers register thousands of "disposable" domains daily. This module contextualizes the domain within the broader DNS ecosystem.

7.1 Implementation Logic: WHOIS and RDAP

The module utilizes **WHOIS** protocols (port 43) or the newer **RDAP** (Registration Data Access Protocol) via JSON APIs to fetch registration metadata.

1. Domain Extraction:
   Using tldextract, the system isolates the effective second-level domain (e.g., google from images.google.co.uk) to perform the lookup on the correct registry entity.9
2. Age Analysis:
   The creation_date is extracted from the WHOIS record.
   - *Logic:* Domain Age = Current Date - Creation Date.
   - *Risk Thresholds:* Domains created < 1 week ago are flagged as "Critical Risk" (New Domain). Domains < 1 month are "High Risk." This effectively catches "snowshoe" spam and fresh phishing infrastructure.[1]
3. Suspicious TLD Detection:
   The Top-Level Domain (TLD) is compared against a weighted list of TLDs known for high abuse rates and low registration costs (e.g., .xyz, .top, .tk, .gq, .zip). While a TLD alone doesn't prove malice, a .zip domain combined with a recent registration date is a potent threat signal.1
4. Registrar Reputation:
   The registrar field is analyzed. Certain registrars are known to have lax abuse policies. Identifying a domain registered via a "bulletproof" registrar adds to the risk score.

7.2 Potential Problems and Mitigations

- **Problem:** *WHOIS Privacy and Redaction.* GDPR and similar privacy laws mean registrant data is often redacted ("Redacted for Privacy").
  - *Mitigation:* The system focuses on the *creation date* and *registrar*, which are

generally still visible even in redacted records. If the creation date is missing, the system treats the domain age as "Unknown" (Indeterminate), adjusting the confidence of the risk score.[26]

- **Problem:** *Rate Limiting.* Registrars aggressively rate-limit WHOIS queries.
  - *Mitigation:* Implement caching using Redis to store WHOIS data for 24 hours. For high-volume deployments, integrate with a commercial WHOIS API service or a rotating proxy pool.[27]

---

8. Module VI: Advanced Lexical Analysis and Phishing Detection

This module focuses on the "look and feel" of the URL text itself. Phishing relies heavily on visual deception (Typosquatting) and semantic deception (Keywords).

8.1 Algorithms and Detection Logic

1. Typosquatting Detection (Edit Distance):
   The system maintains a reference list of high-value targets (e.g., "google", "facebook", "amazon", "paypal", "microsoft").
   - *Algorithm:* Calculate the **Levenshtein Distance** between the analyzed domain and the target list. If the distance is 1 or 2 (indicating one or two character changes, insertions, or deletions), it flags the domain as a potential typosquat (e.g., gooogle.com).[7]
   - *Jaro-Winkler Distance:* Alternatively, use Jaro-Winkler distance which gives higher weight to prefix matches, useful for detecting subdomain spoofing.
2. Keyword Analysis:
   Attackers inject trust-inducing words into the URL path or subdomains to lower victim suspicion.
   - *Logic:* Scan the URL for a dictionary of high-risk keywords: login, secure, account, verify, update, support, billing. A URL like paypal-secure.com or apple.verify-id.com triggers a "Suspicious Keyword" flag.[1]
3. Entropy Calculation (DGA Detection):
   Malware Command & Control (C2) servers often use Domain Generation Algorithms (DGA) to create random domain names (e.g., xkzj194.com).
   - Algorithm: Calculate the Shannon Entropy of the domain string.

     $$H(X) = - \sum_{i=1}^{n} P(x_i) \log_2 P(x_i)$$
   - *Logic:* Domains with high entropy (indicating a random distribution of characters) are flagged as potential DGA domains.[29]

8.2 Potential Problems and Mitigations

- **Problem:** *False Positives in Entropy.* Legitimate domains like cdn-x84-y92.aws.com can have high entropy.
  - *Mitigation:* Whitelist known CDN providers (e.g., AWS, Akamai) to prevent entropy checks from flagging infrastructure domains.
- **Problem:** *Performance of Edit Distance.* Calculating Levenshtein distance against a massive list of brands is slow ($O(N \times M)$).
  - *Mitigation:* Use optimized libraries like python-Levenshtein (C-based) and limit checks to the top 50-100 most phished brands rather than the entire Fortune 500.

---

9. Module VII: Machine Learning Risk Prediction Engine

While heuristic rules capture known patterns, they fail against novel attacks. The Machine Learning module provides a probabilistic assessment, aggregating subtle features that rules might miss.

9.1 Feature Engineering

The raw URL is transformed into a numerical feature vector. Based on academic literature [8], the following features are extracted:

- **Lexical Features:** Length of URL, length of domain, count of dots/hyphens/digits, presence of @ (URL masking), depth of path hierarchy.
- **Statistical Features:** Shannon entropy of the domain and path.
- **Host-Based Features (from previous modules):** Domain age (in days), Certificate Validity (Binary), Redirect Count (Integer), Is_HTTPS (Binary).

9.2 Model Specification: Random Forest Classifier

The **Random Forest** algorithm is specified for this module.

- *Justification:* Unlike deep neural networks, Random Forests perform exceptionally well on tabular data with relatively small datasets. They are resistant to overfitting and, crucially, provide feature importance scores, allowing the system to explain *why* a URL was flagged (e.g., "70% contribution from Domain Age").[3]
- *Training Data:* The model is trained on balanced datasets comprising malicious URLs (from PhishTank, OpenPhish) and benign URLs (from Alexa/Tranco Top 1M).[34]

9.3 Implementation Logic

1. **Vectorization:** Convert the current URL into the feature array.
2. **Inference:** Pass the vector to the pre-trained Scikit-learn model using model.predict_proba().
3. **Output:** The model returns a probability score (0.0 to 1.0) indicating the likelihood of malicious intent. This probability feeds directly into the final risk score calculation.[5]

9.4 Potential Problems and Mitigations

- **Problem:** *Model Drift*. Attackers change tactics (e.g., moving from long to short URLs), making the model obsolete.
  - *Mitigation:* Implement a feedback loop where false positives/negatives reported by users or Tier 2 scans are added to the training set for periodic re-training (MLOps pipeline).

---

10. Module VIII: Static Content Inspection for Insecure Elements

This module acts as a "lightweight parser" for the content retrieved during the redirect trace. It looks for gross security violations without rendering the page.

10.1 Logic: Detecting Insecure Login Forms

Phishing sites often host login forms on HTTP pages or submit data to HTTP endpoints, exposing credentials to interception.

- *Library:* **BeautifulSoup (bs4)** is used for parsing the HTML text.[36]
- *Algorithm:*
  1. Parse the HTML body.
  2. Search for <input> tags where type="password".
  3. If found, inspect the parent <form> tag's action attribute.
  4. **Violation Check:** If the page URL is http:// OR the form action is http://, flag immediately as "Insecure Login Form." This is a high-confidence indicator of a poorly secured or malicious site.[1]

10.2 Potential Problems and Mitigations

- **Problem:** *Dynamic Forms*. Forms generated purely by JavaScript (React/Angular apps) will not be visible in the static HTML source.
  - *Mitigation:* This is a known limitation. The system notes "Dynamic Content Detected" if <script> tags are prevalent but no forms are found, hinting that Tier 2 analysis might be necessary.

---

11. Advanced Risk Scoring Algorithm and Fusion Logic

The individual outputs from the eight modules must be synthesized into a single, actionable Risk Score (0-100). A simple average is insufficient; a "New Domain" might be benign, but

"New Domain + Login Keyword" is critical.

11.1 The Risk Formula

The system uses a **Weighted Risk Fusion** model. It combines a baseline probabilistic score from the ML model with additive penalties from heuristic checks, capped by "Override" signals for critical threats.[22]

Let S_{ML} be the ML probability (0-100).
Let P_i be the penalty for heuristic factor i.
Let W_i be the weight of factor i.

$$\text{Risk Score} = \min \left(100, \left(\alpha \cdot S_{ML} \right) + \sum (P_i \times W_i) \right)$$
If a **Critical Indicator** (e.g., Phishing Blacklist Hit) is found, the score is forced to 100 immediately.

11.2 Weights and Penalties Table

Detected Signal	Severity	Penalty Points	Reasoning
Domain Age < 7 Days	High	+40	High likelihood of burner domain.
Typosquatting Match	High	+50	Intentional deception detected.
Insecure Login (HTTP)	Critical	+100 (Override)	Immediate credential theft risk.
Suspicious Keyword	Medium	+20	Contextual risk.
DV SSL Certificate	Low	+10	Low trust verification.
Cross-Domain Redirect	Low	+15	Potential cloaking.

ML Probability > 80%	High	+30	Statistical pattern match.

11.3 Indeterminate Attribute Handling

If a check fails (e.g., WHOIS timeout), it is treated as "Indeterminate." The scoring algorithm normalizes the remaining weights to ensure the score reflects only the known data, preventing artificial deflation of the risk score due to missing info.[26]

---

12. Result Aggregation and User Presentation Layer

The final output is a structured JSON object tailored for the React frontend to display an "at-a-glance" dashboard.

12.1 Result Summary Components

The display logic categorizes the Risk Score into three tiers [1]:

1. **Safe (0-30):** Green Shield. "Safe to Visit."
2. **Caution (31-69):** Yellow/Orange Warning. "Proceed with Caution." (UI suggests running Tier 2 Deep Scan).
3. **High Risk (70-100):** Red Alert. "Dangerous Link Detected." (UI blocks the link or requires explicit override).

12.2 Explainability Features

Crucially, the result includes a "Reasoning" section. Instead of a black-box verdict, it lists specific triggers:

- *"Domain registered only 3 days ago."*
- *"Contains login keywords but uses a low-trust certificate."*
- *"Redirects through 3 different domains."*

This transparency builds user trust and educates them on *why* the link was flagged.

---

13. Operational Challenges, Mitigations, and Future Scalability

13.1 Scalability and Caching

As traffic grows, sequentially running these checks becomes a bottleneck.

- **Solution:** Use **Celery** with **Redis** as a message broker. When a URL is submitted, tasks (WHOIS, SSL, ML) are dispatched to worker queues in parallel. The results are aggregated as they return.
- **Caching:** Redis is also used to cache the analysis results of URLs. If user B submits a link that User A scanned 5 minutes ago, the result is returned instantly from the cache, bypassing all modules.[37]

13.2 Rate Limiting Defense

To prevent SafeNav from being blocked by external WHOIS or DNS providers:

- **Solution:** Implement a **Rotating Proxy Pool** for egress traffic.
- **Solution:** Adhere to "polite" crawling standards (e.g., 1 request per second per TLD) managed by a token bucket algorithm in the application logic.[27]

13.3 Conclusion

Phase 1 of SafeNav represents a comprehensive, high-velocity security barrier. By intelligently layering syntactic normalization, heuristic taxonomy, cryptographic validation, and machine learning, it achieves the capability to filter the vast majority of web threats in real-time. This architecture minimizes the load on the subsequent Deep Scan phase, ensuring a system that is both economically viable and operationally robust in the face of evolving cyber threats.

Citations: [1]

Works cited

1. SAFENAV.docx
2. urllib.parse — Parse URLs into components — Python 3.14.2 documentation, accessed January 20, 2026, https://docs.python.org/3/library/urllib.parse.html
3. Real-Time Detection of Malicious URLs Using Feature-Based Machine Learning Approaches - IJIRMPS, accessed January 20, 2026, https://www.ijirmps.org/papers/2025/2/232401.pdf
4. HTTPS redirect does not exist in FastAPI/Starlette or infinite loop - Reddit, accessed January 20, 2026, https://www.reddit.com/r/FastAPI/comments/1ffxwps/https_redirect_does_not_exist_in_fastapistarlette/
5. Phishing URL Detection: A Network-based Approach Robust to Evasion - arXiv, accessed January 20, 2026, https://arxiv.org/pdf/2209.01454
6. tessl/pypi-url-normalize@2.2.x - Registry, accessed January 20, 2026, https://tessl.io/registry/tessl/pypi-url-normalize
7. BUILDING A PHISHING URL SCANNER IN PYTHON. | by Heeruzen - Medium, accessed January 20, 2026, https://medium.com/@heeruzn/building-a-phishing-url-scanner-in-python-eae78

cf0e12c
8. Malicious URL Detection with Advanced Machine Learning and Optimization-Supported Deep Learning Models - MDPI, accessed January 20, 2026, https://www.mdpi.com/2076-3417/15/18/10090
9. tldextract · PyPI, accessed January 20, 2026, https://pypi.org/project/tldextract/
10. How to extract top-level domain name (TLD) from URL - Stack Overflow, accessed January 20, 2026, https://stackoverflow.com/questions/1066933/how-to-extract-top-level-domain-name-tld-from-url
11. Static Analysis of Deeplinks and App Links in Mobile Apps | by Tushar Saini | Medium, accessed January 20, 2026, https://medium.com/@tusharsaini484/static-analysis-of-deeplinks-and-app-links-in-mobile-apps-6cefafbc4d48
12. Configure, implement, and verify Android App Links, accessed January 20, 2026, https://developer.android.com/codelabs/android-app-links-introduction
13. How to Easily Send HTTP Requests That Mimic a Browser : r/Python - Reddit, accessed January 20, 2026, https://www.reddit.com/r/Python/comments/1feo3jo/how_to_easily_send_http_requests_that_mimic_a/
14. How does the python requests library send http requests without a browser ? what mechanism does it use ? and does not using a browser have a downside ? : r/HowToHack - Reddit, accessed January 20, 2026, https://www.reddit.com/r/HowToHack/comments/iq7bo2/how_does_the_python_requests_library_send_http/
15. How to trace or to check history of redirected URLs with python only urllib library, accessed January 20, 2026, https://stackoverflow.com/questions/47002795/how-to-trace-or-to-check-history-of-redirected-urls-with-python-only-urllib-libr
16. Python Requests library redirect new url - Stack Overflow, accessed January 20, 2026, https://stackoverflow.com/questions/20475552/python-requests-library-redirect-new-url
17. How to Follow Redirects With Python Requests? - Oxylabs, accessed January 20, 2026, https://oxylabs.io/resources/web-scraping-faq/python/requests-redirects
18. How to Python requests to follow URL like my browser - Stack Overflow, accessed January 20, 2026, https://stackoverflow.com/questions/41352373/how-to-python-requests-to-follow-url-like-my-browser
19. ssl — TLS/SSL wrapper for socket objects — Python 3.14.2 documentation, accessed January 20, 2026, https://docs.python.org/3/library/ssl.html
20. CheckSSLCert - PyPI, accessed January 20, 2026, https://pypi.org/project/CheckSSLCert/
21. DV, OV, & EV SSL certificate validation levels explained - Security Boulevard, accessed January 20, 2026, https://securityboulevard.com/2024/11/dv-ov-ev-ssl-certificate-validation-levels-

explained/
22. 7 Methods For Calculating Cybersecurity Risk Scores - Centraleyes, accessed January 20, 2026, https://www.centraleyes.com/7-methods-for-calculating-cybersecurity-risk-scores/
23. nabla-c0d3/sslyze: Fast and powerful SSL/TLS scanning library. - GitHub, accessed January 20, 2026, https://github.com/nabla-c0d3/sslyze
24. iphelix/sslmap: SSLMap - TLS/SSL cipher suite scanner. - GitHub, accessed January 20, 2026, https://github.com/iphelix/sslmap
25. Automating the Detection of Malicious URLs : r/cybersecurity - Reddit, accessed January 20, 2026, https://www.reddit.com/r/cybersecurity/comments/192llqy/automating_the_detection_of_malicious_urls/
26. Risk score calculation - IBM, accessed January 20, 2026, https://www.ibm.com/docs/en/sva/11.0.0?topic=overview-risk-score-calculation
27. WHOIS rate limiting : OpenSRS Customer Support, accessed January 20, 2026, https://support.opensrs.com/support/solutions/articles/201000063438-whois-rate-limiting
28. Handling API Rate Limits with Python: A Simple Recursive Approach - Medium, accessed January 20, 2026, https://medium.com/@balakrishnamaduru/handling-api-rate-limits-with-python-a-simple-recursive-approach-08349dd71057
29. Malicious URL Detection Based on Associative Classification - MDPI, accessed January 20, 2026, https://www.mdpi.com/1099-4300/23/2/182
30. Using Entropy in Threat Hunting: a Mathematical Search for the Unknown - Red Canary, accessed January 20, 2026, https://redcanary.com/blog/threat-detection/threat-hunting-entropy/
31. (PDF) Combining Lexical, Host, and Content-based features for Phishing Websites detection using Machine Learning Models - ResearchGate, accessed January 20, 2026, https://www.researchgate.net/publication/379902411_Combining_Lexical_Host_and_Content-based_features_for_Phishing_Websites_detection_using_Machine_Learning_Models
32. An Assessment of Lexical, Network, and Content-Based Features for Detecting Malicious URLs Using Machine Learning and Deep Learning Models - NIH, accessed January 20, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC9436524/
33. Using Lexical Features for Malicious URL Detection - A Machine Learning Approach - arXiv, accessed January 20, 2026, https://arxiv.org/pdf/1910.06277
34. Using Lexical Features for Malicious URL Detection -- A Machine Learning Approach - arXiv, accessed January 20, 2026, https://arxiv.org/abs/1910.06277
35. Phishing URLs Detection Using Sequential and Parallel ML Techniques - NIH, accessed January 20, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10098541/
36. Attacking Web Applications With Python: Exploiting Web Forms and Requests - Infosec, accessed January 20, 2026, https://www.infosecinstitute.com/resources/secure-coding/attacking-web-applic

ations-with-python-exploiting-web-forms-and-requests/
37. Trace all your HTTP requests in Python using HTTPDBG | by cle-b - Medium, accessed January 20, 2026, https://medium.com/@cle-b/trace-all-your-http-requests-in-python-using-httpdbg-9db9110db981
38. paarthbhatt/WhoIs-Lookup-Terminal-Tool - GitHub, accessed January 20, 2026, https://github.com/paarthbhatt/Whois-Lookup-Terminal-Tool
39. medialab/ural: A helper library full of URL-related heuristics. - GitHub, accessed January 20, 2026, https://github.com/medialab/ural
40. The Best Python HTTP Clients (Python Requests Alternatives) - Proxyway, accessed January 20, 2026, https://proxyway.com/guides/the-best-python-http-clients
41. ipwhois · PyPI, accessed January 20, 2026, https://pypi.org/project/ipwhois/
42. ipwhois 1.3.0 documentation, accessed January 20, 2026, https://ipwhois.readthedocs.io/en/latest/README.html
43. Static and Dynamic Analysis: Essential Tools for Software Security - Medium, accessed January 20, 2026, https://medium.com/@10086_54997/static-and-dynamic-analysis-essential-tools-for-software-security-4ba71bf49eaf